ChatGPT

Siemens NX 2306 Assembly Section Crosshatch Automation by Material or Part Number

Recommended design and feasibility findings

The most practical NX 2306 solution is **not to generate detached hatch geometry yourself**. Let NX create and maintain the section view, section boundaries, and native hatch objects; your NX Open application should identify which component owns each sectioned region and then **edit the style of the existing NX hatch**. That architecture gives NX responsibility for geometric associativity when components move or the section definition changes, while your application owns classification, attribute resolution, rule selection, validation, and styling. NX exposes a `HatchCollection` and `CreateHatchBuilder(Hatch)` in the SC2306 managed API, and the hatch settings API includes pattern, angle, distance, scale, crosshatch file, and a specific `ApplyToAllFromSameComp` setting. [1]

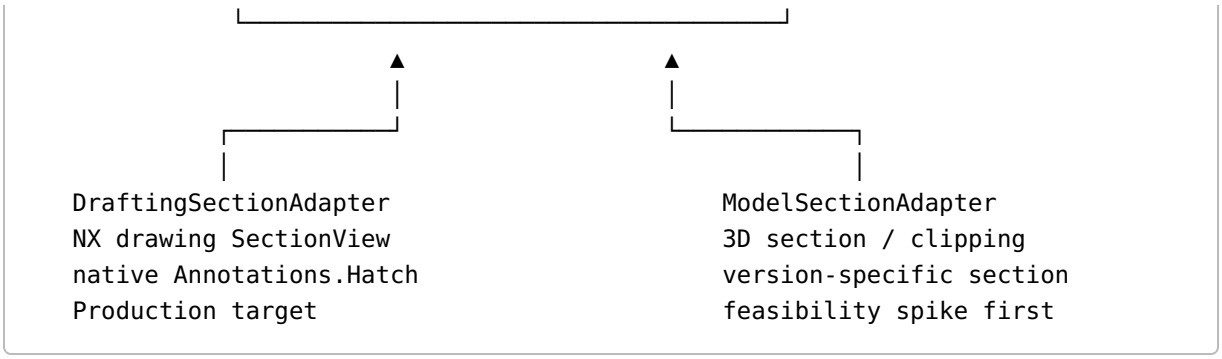
Siemens has supported component-oriented crosshatch handling in section drawings for many NX releases: its crosshatching enhancement documentation describes changing crosshatch parameters for a component across multiple section views. That is important because it confirms that **component identity is a meaningful concept in NX's native crosshatch workflow**, rather than something that must be simulated entirely by screen-space geometry. [2]

For NX Open, the assembly object model also contains the pieces needed for ownership resolution. In SC2306, an `NXObject` can expose `IsOccurrence`, `OwningComponent`, `OwningPart`, and `Prototype`; `Component` exposes `Parent`, `GetChildren()`, `FindOccurrence()`, instance attributes, and occurrence position/orientation. Those APIs are sufficient to build a leaf-component ownership map and retain full nested-subassembly paths. [3]

There is one important scope distinction:

2D drafting section views are the strongest candidate for a production implementation in NX 2306. NX Open exposes native drafting `Hatch` objects and section-view infrastructure directly. The public material I found is considerably less explicit about modifying the visual crosshatch of an interactive **3D modeling section** through the same annotation API. I therefore recommend a shared core with two adapters:

```
                Shared automation core
        ┌──────────────────────────────────────┐
        │ Assembly ownership                   │
        │ Attribute resolver                   │
        │ Material / P/N rule engine           │
        │ Conflict detection                   │
        │ Validation + audit                   │
```

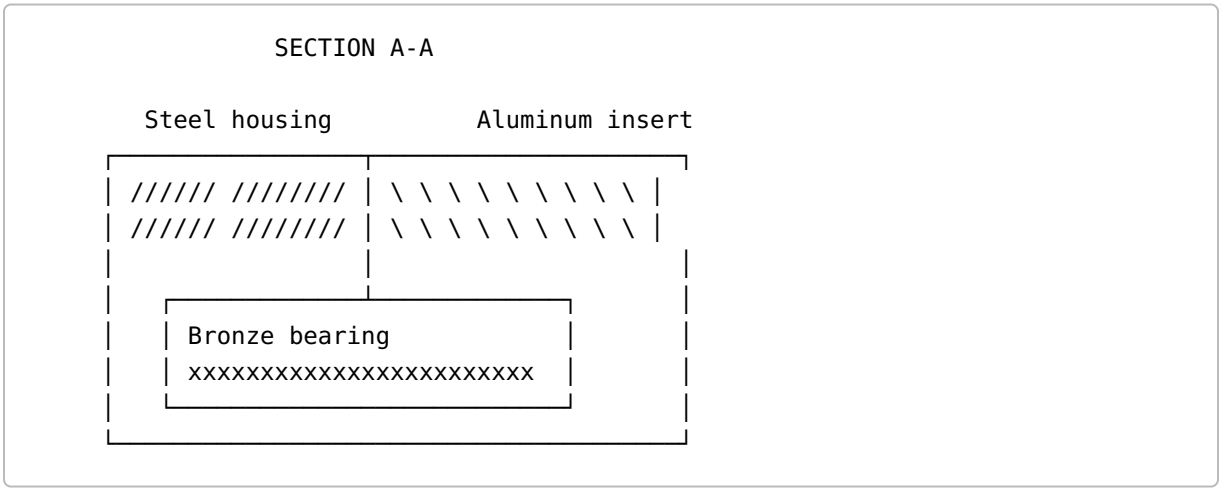


The **first production release should therefore target assembly drawing section views**, while the 3D adapter should be qualified by recording the exact NX 2306 interactive 3D-section workflow and checking which generated objects remain exposed to NX Open. This avoids falsely assuming that `Annotations.Hatch` represents every kind of 3D sectional display.

A second important finding is that the system should use **topological ownership first and ray casting only as a fallback**. A section region belongs to a sectioned solid; the strongest mapping therefore follows NX's section topology back to the original body/component. A long-standing UF_DRAW implementation pattern uses section solids, section edges, and `AskSolidOfSection` to recover the originating `Body`, after which `OwningComponent` gives assembly ownership. The example is older and must be compile-tested in NX 2306, but its topology-first principle is much stronger than guessing ownership from hatch centroid alone. [4]

Target behavior

For an assembly such as:



the application should produce an ownership model like:

```
HATCH-001
  → section solid SX-42
  → occurrence body
```

```
  → ASSY_TOP/HSG_01
  → P/N 10024567
  → Material STEEL_4140
  → rule MAT_STEEL
  → pattern STEEL_STD, 45°, spacing 2.0

HATCH-002
  → section solid SX-47
  → occurrence body
  → ASSY_TOP/SUB_ASSY_02/INSERT_03
  → P/N 20078412
  → Material AL_6061_T6
  → rule MAT_ALUMINUM
  → pattern AL_STD, 135°, spacing 2.5

HATCH-003
  → section solid SX-51
  → occurrence body
  → ASSY_TOP/SUB_ASSY_02/BRG_01
  → P/N BRG-6204
  → Material BRONZE
  → rule PN_BRG
  → pattern BRONZE_STD
```

The important point is that the **displayed hatch is the output, not the source of truth**.

NX capabilities, licensing, and data contract

Required NX capabilities and licenses

Exact Siemens license feature names vary with customer bundles and contract configuration, so this should be verified against your NX 2306 installation before development. The following is the practical capability matrix.

Capability	Need	Purpose in this solution	Practical note
NX Drafting	Required for drawing implementation	Assembly section views, section geometry, native drawing crosshatch	Hatch/view editing belongs to the drafting workflow. Siemens documentation and NX Open APIs expose crosshatch as drafting functionality. [5]

Capability	Need	Purpose in this solution	Practical note
NX Assemblies	Required	Traverse component occurrences, nested subassemblies, suppression and occurrence ownership	SC2306 `Component` exposes `Parent`, `GetChildren`, `FindOccurrence`, suppression state and occurrence data. [6]
NX Open .NET / customization environment	Required for development	C# plug-in/journal, rules, UI, logging	Siemens describes NX Open for .NET Author as the environment/API tooling used to create custom applications; deployment/signing requirements should be confirmed for your licensed environment. [7]
UFUN access through NX Open	Strongly recommended	Section-solid/section-edge topology queries, geometric fallback utilities	Use the UF layer where the higher-level managed API does not expose section topology conveniently.
Block UI Styler / NX dialog framework	Recommended	Review table, conflict resolution, highlight/apply workflow	Prefer a dockable or modal review tool rather than silently changing released drawings.
Teamcenter Integration for NX	Optional	Obtain managed P/N, revision, BOM-line or mapped material properties	Teamcenter-to-NX values depend on site attribute mapping; Siemens has published NX 2306-specific attribute-mapping troubleshooting. [8]
Teamcenter Integrated Material Management	Optional	Use Teamcenter material as the authoritative material source	Siemens states that its NX extension can assign material to CAD models and synchronize material information between Teamcenter and NX. [9]

NX part attributes are a suitable integration layer because Teamcenter supports NX part-attribute content and Siemens explicitly describes searching/filtering NX models using NX part attributes. [10]

Proposed attribute contract

Do not make the C# application depend directly on one hard-coded material attribute name. Teamcenter/NX attribute mappings are site-specific, and even Siemens' NX 2306 Teamcenter support material documents situations where mapped values require additional configuration. [8]

Instead, give the resolver a configurable list of candidate attributes and an explicit precedence model.

Attribute	Scope	Example	Role
`DB_PART_NO`	Prototype part, Teamcenter-managed	`10024567`	Preferred managed part number where your site maps it
`DB_PART_REV`	Prototype part	`C`	Revision/audit identity
`MATERIAL`	Prototype part	`AISI 4140`	Common local/material text candidate; configure rather than assume
`MATERIAL_ID`	Prototype part	`MAT-STEEL-4140`	Optional controlled material identifier
`MBD_MATERIAL_CLASS`	Prototype part	`STEEL`	**Proposed custom** normalized class for automation
`MBD_HATCH_CLASS`	Prototype or occurrence	`FERROUS`	**Proposed custom** direct style-class override
`MBD_HATCH_PATTERN`	Prototype or occurrence	`STEEL_STD`	**Proposed custom** explicit pattern override
`MBD_HATCH_ANGLE`	Prototype or occurrence	`45`	**Proposed custom** explicit angle override
`MBD_HATCH_SPACING`	Prototype or occurrence	`2.0`	**Proposed custom** explicit spacing override
`MBD_HATCH_SCALE`	Prototype or occurrence	`1.0`	**Proposed custom** scale override
`MBD_HATCH_LOCK`	Hatch/occurrence	`TRUE`	**Proposed custom** preserve manual styling
`AUTO_HATCH_MANAGED`	Hatch	`TRUE`	**Proposed audit attribute** identifies automation-managed style
`AUTO_HATCH_RULE_ID`	Hatch	`MAT_STEEL_V3`	**Proposed audit attribute** last applied rule
`AUTO_HATCH_STYLE_HASH`	Hatch	`70B3…`	**Proposed audit attribute** idempotency/manual-edit detection
`AUTO_HATCH_OWNER_KEY`	Hatch	`TOP/SUB1/HSG_2`	**Proposed audit attribute** expected occurrence identity

`DB_PART_NO` and other `DB_*` fields should be treated as **read-only integration data unless your Teamcenter administrators explicitly define otherwise**. The purpose of this application is to consume managed metadata, not rewrite Teamcenter's product identity.

For occurrence-specific behavior, SC2306 exposes instance user attributes directly through methods such as `GetInstanceStringUserAttribute`, allowing two occurrences of the same prototype part to have different hatch overrides without modifying the prototype. [11]

Attribute and rule precedence

I recommend this deterministic order:

```
Highest authority
    |
    |  Occurrence-level explicit MBD_HATCH_* override
    |  Prototype explicit MBD_HATCH_* override
    |  Controlled Teamcenter/NX material classification
    |  Local NX material classification
    |  Part-number-specific rule
    |  Part-number family rule
    |  Company default
    v
Lowest authority
```

The system should **not silently choose between equal-priority contradictory rules**. It should create a conflict record such as:

```
Component:
  TOP_ASSY/SUB_A/PLATE_7

P/N:
  123-7844

Material:
  ALUMINUM_6061

Material rule:
  MAT_AL -> ALUMINUM_STD / 45°

Part-number rule:
  PN_123_78XX -> STEEL_STD / 45°

Status:
  CONFLICT
```

```
Required action:
  Engineer selects authoritative rule or adds controlled override.
```

A simple external rules file keeps engineering policy out of the compiled plug-in:

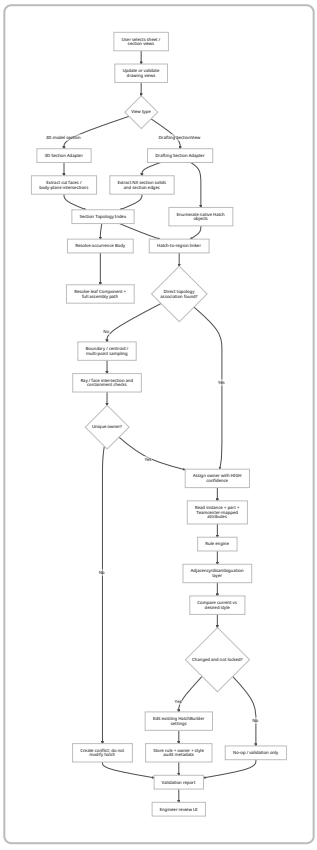
```
{
  "schemaVersion": "1.0",
  "ruleSetId": "NX2306_SECTION_HATCH_V4",
  "attributePrecedence": [
    "INSTANCE_OVERRIDE",
    "PART_OVERRIDE",
    "MATERIAL",
    "PART_NUMBER",
    "DEFAULT"
  ],
  "materialRules": [
    {
      "id": "MAT_STEEL",
      "match": "^STEEL.*|^AISI.*",
      "pattern": "STEEL_STD",
      "angleDeg": 45.0,
      "spacingMm": 2.0,
      "scale": 1.0,
      "priority": 100
    },
    {
      "id": "MAT_ALUMINUM",
      "match": "^AL.*|^ALUMINUM.*",
      "pattern": "AL_STD",
      "angleDeg": 45.0,
      "spacingMm": 2.5,
      "scale": 1.0,
      "priority": 100
    }
  ],
  "partNumberRules": [
    {
      "id": "PN_BEARING",
      "regex": "^BRG-",
      "pattern": "BEARING_STD",
      "angleDeg": 135.0,
      "spacingMm": 1.5,
      "priority": 150
    }
  ],
  "defaultStyle": {
```

```
    "pattern": "GENERAL",
    "angleDeg": 45.0,
    "spacingMm": 2.5,
    "scale": 1.0
  }
}
```

The actual `pattern` identifiers need to match the crosshatch definitions installed at your site. The NX hatch settings API exposes both a `CrosshatchFile` and `Pattern`, as well as `FindCrosshatchFile`, so the application should resolve configured identifiers through NX rather than hard-coding an installation-directory path. [12]

Processing architecture and region-to-component mapping

Architecture



Preferred mapping hierarchy

A robust implementation should attempt ownership resolution in the following order.

Direct section topology

For a drafting section view:

1. Identify all **NX section solids** contributing to that section view.
2. For every section solid, enumerate its generated section edges.
3. Resolve each section solid back to its original model solid/body.
4. Resolve that occurrence body's `OwningComponent`.
5. Build an index:

```
SectionView
  SectionSolid A
      → Model Body
      → Component TOP/SUB1/PART_A
      → section edges [e1, e2, e3, ...]

  SectionSolid B
      → Model Body
      → Component TOP/SUB2/PART_B
      → section edges [e4, e5, e6, ...]
```

A long-established NX Open/UF_DRAW implementation pattern uses:

```
AskSxsolidsOfSxview
AskSxedgesOfSxsolid
AskSolidOfSection
Body.OwningComponent
```

to perform exactly this type of section-edge-to-component resolution. Because the published example predates NX 2306 and includes comments about later DraftingCurve changes, **the algorithm is valuable but the exact managed signatures must be proven in your NX 2306 environment rather than copied blindly**. [4]

This is why the first development spike should verify those UF calls on one NX 2306 assembly section view.

Hatch associativity and section-edge tracing

Every hatch is an annotation. The annotation API exposes the number of associations, `GetAssociativity()`, `GetViews()`, and an `IsOutOfDate` state. Use those associations first to see whether an existing generated hatch points directly or indirectly to a section/boundary object belonging to the indexed section-solid topology. [13]

Conceptually:

```
Hatch H1
  association object
      ↓
section boundary / section curve
      ↓
SectionEdgeToSolid dictionary
      ↓
section solid
      ↓
model body occurrence
      ↓
component
```

If that link resolves uniquely, classify it as **HIGH confidence**.

Boundary overlap and topology tracing

When a generated hatch does not expose a directly useful section edge through its managed association, reconstruct the region from the section-solid edges.

For each candidate section solid:

1. project its section edges into the section view plane;
2. join edges into closed planar loops;
3. classify outer loops and holes;
4. compute the loop area and interior sample points;
5. compare against the hatch's region, boundary data, annotation/view association, or hit-test location;
6. retain only candidates with meaningful overlap.

This handles two adjacent components sharing a common projected boundary better than nearest-centroid logic because the entire loop topology contributes to the decision.

Multi-point containment

Do **not** rely on one centroid. A centroid can lie inside a hole, another component, or outside a concave region.

Generate several interior points, for example:

```
region triangulation
        ↓
largest triangle centroid
+ second-largest triangle centroid
+ inset samples near distinct boundary segments
```

For each sample, test against candidate section solids. NX Open/UF functionality includes point-containment operations for surface-related tests, which can be used as part of the fallback classifier. [14]

Require a strong vote such as:

```
3/3 samples → Component A       HIGH
4/5 samples → Component A       MEDIUM
2/5 A, 2/5 B, 1 none            CONFLICT
```

Ray casting / face intersection

For ambiguous projected regions, cast a short ray along the section normal around the cut plane:

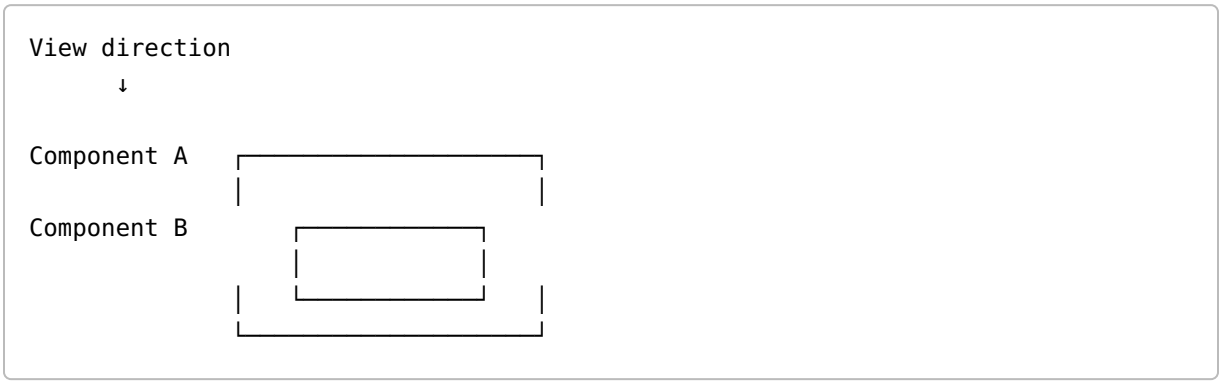
```
                    + section normal
                          ↑
                          |
                ray sample point
                          •
==========================|================ section plane
                          |
                          ↓
                    - section normal
```

Intersect the ray with candidate faces/bodies and determine which occurrence owns the intersection nearest the section plane. Siemens NX Open community guidance points to `UFModl.IntersectCurveToFace` for line/vector-to-face intersection use cases. [15]

This is a **fallback verifier**, not the primary owner resolver.

Why overlapping components require topology rather than screen coordinates

Consider this projected section:



In 2D, a point may be contained by the projection of both components. A simple "point under hatch" query cannot reliably tell which cut region created a particular hatch.

The section-solid topology does. Therefore the owner decision should be:

```
NX section-solid origin
      > exact section-edge topology
      > region-loop match
      > multi-point containment
      > ray casting
      > never arbitrary nearest component
```

Where two occurrence bodies are truly coincident on the section plane and NX does not expose enough topology to distinguish their hatch region, mark the item **AMBIGUOUS_OVERLAP** and leave the existing style unchanged.

Nested subassemblies

When a body occurrence resolves to a leaf `Component`, retain its whole parent chain. SC2306 exposes `Component.Parent`, so an occurrence key can be constructed without confusing identical part prototypes appearing multiple times. [6]

Example:

```
Prototype:
  PLATE_001.prt

Occurrence 1:
  TOP/SUB_LEFT/PLATE_001[1]

Occurrence 2:
  TOP/SUB_RIGHT/PLATE_001[1]
```

These must be treated as separate occurrences even though they share a prototype part and therefore probably share the same prototype material.

A stable logical key could be:

```
TopAssemblyRevision
+
OccurrencePath
+
PrototypePartNumber
```

Do not use NX `Tag` as the long-term identity because tags are session/object identities rather than a durable engineering key.

Adjacent components with the same material

Material-based hatching creates a special readability problem:

```
Steel part A       Steel part B
///////////|///////////
///////////|///////////
```

The material rule says both should use the same base pattern, but engineering drawings often need adjacent pieces to remain visually distinguishable.

Solve that with a **presentation disambiguation layer** after material classification:

```
Base material style:
STEEL → STEEL_STD / 45°

Adjacency resolver:
A → STEEL_STD / 45°
B → STEEL_STD / 135°
C → STEEL_STD / 45° with alternate spacing
```

The chosen variant must be deterministic. A good algorithm is:

1. build an adjacency graph from section loops;
2. group touching regions sharing the same base style;
3. graph-color each group using permitted angle/spacing variants;
4. choose variants using the stable occurrence key so a rerun does not randomly swap angles.

Siemens' hatch settings API exposes angle, distance and scale, which makes this a straightforward post-processing layer. [12]

NX Open C# implementation

The following snippets are intentionally organized as **production building blocks rather than one giant journal**. The SC2306 component and hatch-collection signatures below are backed by the SC2306 generated API reference. Some UF_DRAW section-topology signatures should still be compile-verified against the `NXOpen.UF.dll` shipped with your NX 2306 installation because the public example available for those calls is older. [16]

Core data model

```
using System;
using System.Collections.Generic;
using NXOpen;
using NXOpen.Annotations;
using NXOpen.Assemblies;
using NXOpen.Drawings;
using NXOpen.UF;

namespace Company.Nx.SectionHatch
{
    public enum MappingConfidence
    {
        High,
        Medium,
        Low,
        Conflict
    }

    public sealed class ComponentIdentity
    {
        public Component? Occurrence { get; init; }
        public Part? PrototypePart { get; init; }

        public string OccurrencePath { get; init; } = "";
        public string PartNumber { get; init; } = "";
        public string Revision { get; init; } = "";
        public string Material { get; init; } = "";

        public string StableKey =>
            $"{OccurrencePath}|{PartNumber}|{Revision}";
    }

    public sealed class HatchStyle
    {
        public string RuleId { get; init; } = "";
        public string Pattern { get; init; } = "";
        public string? CrosshatchFileId { get; init; }

        public double AngleDeg { get; init; }
        public double Spacing { get; init; }
        public double Scale { get; init; } = 1.0;

        public string StyleHash { get; init; } = "";
    }
```

```
    public sealed class HatchRegionRecord
    {
        public Hatch Hatch { get; init; } = null!;
        public DraftingView View { get; init; } = null!;

        public ComponentIdentity? Owner { get; set; }
        public MappingConfidence Confidence { get; set; }

        public HatchStyle? ExistingStyle { get; set; }
        public HatchStyle? ProposedStyle { get; set; }

        public string Status { get; set; } = "";
        public string Diagnostic { get; set; } = "";
    }
}
```

Enumerate section views and hatches

SC2306's `HatchCollection` is enumerable and also provides `ToArray()`, so use the collection rather than cycling every drafting entity in the database. [17]

```
using System.Collections.Generic;
using System.Linq;
using NXOpen;
using NXOpen.Annotations;
using NXOpen.Drawings;

static IEnumerable<Hatch> GetHatchesForView(
    Part drawingPart,
    DraftingView targetView)
{
    foreach (Hatch hatch in drawingPart.Annotations.Hatches.ToArray())
    {
        View[] views;

        try
        {
            views = hatch.GetViews();
        }
        catch
        {
            continue;
        }

        if (views.Any(v => v != null && v.Tag == targetView.Tag))
            yield return hatch;
```

```
    }
}
```

`Annotation.GetViews()` and annotation associativity are exposed by the annotation object model. [13]

For view discovery:

```
static IEnumerable<SectionView> GetDrawingSectionViews(Part workPart)
{
    foreach (DraftingView view in workPart.DraftingViews)
    {
        if (view is SectionView sectionView)
            yield return sectionView;
    }
}
```

Depending on how your NX 2306 drawing was constructed, additional specialized section-view subclasses or legacy views may need to be included during qualification.

Build the section-solid ownership index

The following is deliberately marked as a **UFUN adaptation point**. The topology sequence comes from established NX journal practice; compile the exact argument form against the NX 2306 managed UF reference before freezing the wrapper. [4]

```
using NXOpen;
using NXOpen.Assemblies;
using NXOpen.Drawings;
using NXOpen.UF;
using NXOpen.Utilities;
using System.Collections.Generic;

public sealed class SectionSolidInfo
{
    public Tag SectionSolidTag { get; init; }
    public Tag ModelSolidTag { get; init; }

    public Body? BodyOccurrence { get; init; }
    public Component? OwnerComponent { get; init; }

    public HashSet<Tag> SectionEdgeTags { get; } = new();
}

public sealed class SectionTopologyIndex
{
```

```
    public Dictionary<Tag, SectionSolidInfo> BySectionSolid { get; } = new();
    public Dictionary<Tag, SectionSolidInfo> BySectionEdge { get; } = new();
}

static SectionTopologyIndex BuildSectionTopologyIndex(
    UFSession uf,
    SectionView sectionView)
{
    var result = new SectionTopologyIndex();

    /*
     * NOTE:
     * Verify these managed UF_DRAW signatures in your NX 2306
     * NXOpen.UF reference. Different generated language wrappers
     * can express array/output arguments differently.
     */
    Tag[] sxSolidTags = Array.Empty<Tag>();
    int sxSolidCount = 0;

    uf.Draw.AskSxsolidsOfSxview(
        sectionView.Tag,
        null,
        out sxSolidCount,
        out sxSolidTags);

    foreach (Tag sxSolidTag in sxSolidTags)
    {
        Tag modelSolidTag;
        uf.Draw.AskSolidOfSection(sxSolidTag, out modelSolidTag);

        Body? body =
            NXObjectManager.Get(modelSolidTag) as Body;

        Component? component = body?.OwningComponent;

        var info = new SectionSolidInfo
        {
            SectionSolidTag = sxSolidTag,
            ModelSolidTag = modelSolidTag,
            BodyOccurrence = body,
            OwnerComponent = component
        };

        Tag[] sxEdgeTags = Array.Empty<Tag>();
        int sxEdgeCount = 0;

        uf.Draw.AskSxedgesOfSxsolid(
            sxSolidTag,
```

```
            out sxEdgeCount,
            out sxEdgeTags);

        foreach (Tag edgeTag in sxEdgeTags)
        {
            info.SectionEdgeTags.Add(edgeTag);
            result.BySectionEdge[edgeTag] = info;
        }

        result.BySectionSolid[sxSolidTag] = info;
    }

    return result;
}
```

The conceptual relationship is:

```
SectionView
    ↓
AskSxsolidsOfSxview
    ↓
section solid
    ├── AskSxedgesOfSxsolid → generated section edges
    │
    └── AskSolidOfSection → original occurrence Body
                                  ↓
                          Body.OwningComponent
```

Resolve the complete component path

SC2306 exposes both `Parent` and `GetChildren()`, making nested occurrence paths straightforward. [6]

```
static string GetOccurrencePath(Component component)
{
    var names = new Stack<string>();

    Component? current = component;

    while (current != null)
    {
        string name = !string.IsNullOrWhiteSpace(current.DisplayName)
            ? current.DisplayName
            : current.Name;
```

```
        names.Push(name);
        current = current.Parent;
    }

    return string.Join("/", names);
}
```

Also retain `component.Tag` for the current session as a fast lookup key, but not as the durable audit identity.

Resolve an occurrence back to its prototype part

`NXObject.Prototype` and `OwningComponent` are part of the NX object model, and `FindOccurrence` is available where you need to translate prototype geometry into a particular occurrence. [6]

```
static Part? GetPrototypePart(Component component)
{
    return component.Prototype as Part;
}
```

In some assembly-loading modes you may need to ensure the component's prototype is loaded before its attributes can be inspected.

Read occurrence, prototype, and Teamcenter-mapped attributes

Use an attribute resolver that knows **where** a value came from.

```
public sealed record AttributeValue(
    string Name,
    string Value,
    string Source);

static string? TryGetStringAttribute(
    NXObject obj,
    string title)
{
    try
    {
        if (obj.HasUserAttribute(
            title,
            NXObject.AttributeType.String,
            -1))
        {
            return obj.GetStringUserAttribute(title, -1);
        }
```

```
    }
    catch (NXException)
    {
        // Log at debug level.
    }

    return null;
}

static string? TryGetInstanceStringAttribute(
    Component component,
    string title)
{
    try
    {
        return component.GetInstanceStringUserAttribute(
            title,
            -1);
    }
    catch (NXException)
    {
        return null;
    }
}
```

SC2306 exposes `GetInstanceStringUserAttribute`, `GetInstanceUserAttributes`, and the inherited NX-object attribute APIs. [18]

A practical resolver:

```
static AttributeValue? ResolvePartNumber(
    Component component,
    Part prototypePart)
{
    // Occurrence override only if your company allows it.
    string? occurrencePn =
        TryGetInstanceStringAttribute(component, "MBD_PART_NO_OVERRIDE");

    if (!string.IsNullOrWhiteSpace(occurrencePn))
        return new(
            "MBD_PART_NO_OVERRIDE",
            occurrencePn,
            "Occurrence");

    // Teamcenter-mapped candidate.
    string? dbPn =
```

```
        TryGetStringAttribute(prototypePart, "DB_PART_NO");

    if (!string.IsNullOrWhiteSpace(dbPn))
        return new(
            "DB_PART_NO",
            dbPn,
            "Teamcenter/NX mapping");

    // Unmanaged/company attribute.
    string? pn =
        TryGetStringAttribute(prototypePart, "PART_NUMBER");

    if (!string.IsNullOrWhiteSpace(pn))
        return new(
            "PART_NUMBER",
            pn,
            "NX part");

    return null;
}

static AttributeValue? ResolveMaterial(
    Component component,
    Part prototypePart)
{
    string[] occurrenceCandidates =
    {
        "MBD_MATERIAL_CLASS",
        "MBD_MATERIAL_OVERRIDE"
    };

    foreach (string name in occurrenceCandidates)
    {
        string? value =
            TryGetInstanceStringAttribute(component, name);

        if (!string.IsNullOrWhiteSpace(value))
            return new(name, value, "Occurrence");
    }

    string[] partCandidates =
    {
        "MBD_MATERIAL_CLASS",
        "MATERIAL_ID",
        "MATERIAL"
    };

    foreach (string name in partCandidates)
```

```
    {
        string? value =
            TryGetStringAttribute(prototypePart, name);

        if (!string.IsNullOrWhiteSpace(value))
            return new(name, value, "NX/Teamcenter part");
    }

    return null;
}
```

The candidate names beyond your actual Teamcenter mapping are design suggestions; configure them centrally rather than assuming every NX/Teamcenter installation uses identical titles. Siemens explicitly documents that Teamcenter/NX attribute mapping is configurable, while Teamcenter Integrated Material Management can synchronize material information into NX-managed workflows. [19]

Link hatch associations to section topology

Use direct annotation associations before geometric inference:

```
static SectionSolidInfo? TryResolveFromHatchAssociativity(
    Hatch hatch,
    SectionTopologyIndex index)
{
    for (int i = 1; i <= hatch.NumberOfAssociativities; i++)
    {
        Associativity assoc;

        try
        {
            assoc = hatch.GetAssociativity(i);
        }
        catch (NXException)
        {
            continue;
        }

        NXObject?[] objects =
        {
            assoc.FirstObject,
            assoc.SecondObject
        };

        foreach (NXObject? obj in objects)
        {
            if (obj == null)
```

```
                continue;

            if (index.BySectionEdge.TryGetValue(
                obj.Tag,
                out SectionSolidInfo? owner))
            {
                return owner;
            }
        }
    }

    return null;
}
```

This is intentionally the first stage only. Auto-generated section hatches can expose associations through intermediary drafting/boundary objects rather than a raw section-edge tag, so the production mapper should recursively inspect recognized boundary/section-curve objects and then fall back to geometric loop matching when the direct tag test does not resolve.

Fallback face-intersection verifier

A conceptual ray verifier is:

```
// Pseudocode around UFModl.IntersectCurveToFace:
//
// Point3d p0 = samplePoint - sectionNormal * epsilon;
// Point3d p1 = samplePoint + sectionNormal * epsilon;
//
// Line probe = CreateTemporaryLine(p0, p1);
//
// foreach (Face candidateFace in candidateFaces)
// {
//     intersections = UFModl.IntersectCurveToFace(
//         probe.Tag,
//         candidateFace.Tag);
//
//     score candidate ownership according to intersection
//     distance from section plane;
// }
//
// Delete temporary probe.
```

Siemens' NX Open community guidance specifically identifies `UFModl.IntersectCurveToFace` for intersecting a direction/line with a face. [15]

In production, restrict candidate faces to the section-solid owners from the current view. There is little reason to ray-test thousands of unrelated assembly faces after NX itself has already identified which solids participate in the section.

Apply or update an existing hatch style

This is the critical operation. **Edit the native hatch; do not replace it with a manually bounded hatch.**

SC2306 exposes `HatchCollection.CreateHatchBuilder(Hatch)`, while the hatch-fill settings API exposes pattern, file, angle, distance, scale and component-propagation settings. [1]

```
using NXOpen;
using NXOpen.Annotations;

static void ApplyHatchStyle(
    Part drawingPart,
    Hatch hatch,
    HatchStyle style)
{
    HatchBuilder builder =
        drawingPart.Annotations.Hatches.CreateHatchBuilder(hatch);

    try
    {
        HatchFillSettingsBuilder settings =
            builder.HatchFillSettings;

        // Existing section hatch: change appearance only.
        // Do not replace its boundary.
        settings.Pattern = style.Pattern;
        settings.Angle = style.AngleDeg;
        settings.Distance = style.Spacing;
        settings.Scale = style.Scale;

        if (!string.IsNullOrWhiteSpace(style.CrosshatchFileId))
        {
            string resolved =
                settings.FindCrosshatchFile(
                    style.CrosshatchFileId);

            if (!string.IsNullOrWhiteSpace(resolved))
                settings.CrosshatchFile = resolved;
        }

        /*
         * Safer default for the automation:
```

```
            * update the specific mapped hatch only.
            *
            * Set true only where you intentionally want NX to
            * propagate style to all hatch regions belonging to
            * the same component.
            */
           settings.ApplyToAllFromSameComp = false;

           NXObject committed = builder.Commit();

           if (committed == null)
               throw new NXException(
                   "Hatch builder committed without an object.");
       }
       finally
       {
           builder.Destroy();
       }
   }
```

The API exposes `ApplyToAllFromSameComp`; Siemens also describes user-level support for setting component crosshatch parameters across multiple section views. That can be useful, but a rules engine should normally leave it `false` and explicitly update only the hatches it has mapped, because an occurrence may intentionally have different section presentation in another view. [20]

Before finalizing this method, record a manual **Edit Crosshatch** operation in NX 2306 and compare the generated journal against these fields. That verifies your site's actual pattern file and unit conventions.

Do not create a hatch when NX should have generated one

For an assembly section view, a missing section hatch should normally result in:

```
MISSING_NATIVE_HATCH
→ update section view
→ re-query hatch collection
→ if still missing, report validation error
```

not:

```
create arbitrary manual hatch from projected curves
```

The latter severs responsibility for boundary associativity from the section-view engine.

Record automation metadata

Because `Hatch` inherits the NX object attribute infrastructure, store limited audit information on the hatch object:

```
static void StampManagedHatch(
    Hatch hatch,
    ComponentIdentity owner,
    HatchStyle style)
{
    const int scalar = -1;

    hatch.SetUserAttribute(
        "AUTO_HATCH_MANAGED",
        scalar,
        "TRUE",
        Update.Option.Later);

    hatch.SetUserAttribute(
        "AUTO_HATCH_OWNER_KEY",
        scalar,
        owner.StableKey,
        Update.Option.Later);

    hatch.SetUserAttribute(
        "AUTO_HATCH_RULE_ID",
        scalar,
        style.RuleId,
        Update.Option.Later);

    hatch.SetUserAttribute(
        "AUTO_HATCH_STYLE_HASH",
        scalar,
        style.StyleHash,
        Update.Option.Later);
}
```

NX objects expose general user-attribute read/write functionality in the SC2306 API object model. [21]

Also keep a drawing-level audit record, because a view regeneration could theoretically replace a generated annotation object. The durable state should therefore be reconstructible from:

```
view identity
+ component occurrence path
```

```
+ current topology
+ rule-set revision
```

rather than depending on the hatch's NX tag or hatch attributes alone.

Batch transaction

Apply all edits under one visible undo mark:

```
Session session = Session.GetSession();

Session.UndoMarkId mark =
    session.SetUndoMark(
        Session.MarkVisibility.Visible,
        "Apply Material-Based Section Hatching");

try
{
    // Analyze all views first.
    // Resolve all conflicts.
    // Apply only approved records.

    int errors =
        session.UpdateManager.DoUpdate(mark);

    if (errors != 0)
    {
        // Report/update review state.
    }
}
catch
{
    session.UndoToMark(mark, null);
    throw;
}
```

This gives the user a coherent undo operation instead of hundreds of isolated hatch edits.

Associativity, idempotency, conflicts, review UI, and performance

Preserving associativity

The core rule is:

> **NX owns section geometry; the automation owns only style metadata.**

A drawing view can become out of date when the model changes, and Siemens' current NX drawing workflow explicitly represents out-of-date views and provides view/sheet updating. [22]

The supported lifecycle should therefore be:

```
Assembly changes
      ↓
NX marks / updates section view
      ↓
NX regenerates section geometry and native hatch boundaries
      ↓
Automation rescans current section topology
      ↓
Component ownership is recomputed
      ↓
Rules are reapplied to existing hatches
```

That is considerably safer than trying to keep your own section polygons synchronized with moving components.

Recommended command workflow:

```
Section Hatch Manager
    [ Update Views ]
    [ Analyze ]
    [ Review ]
    [ Apply ]
    [ Validate ]
```

For controlled-release environments, make **Analyze + Validate** mandatory before drawing release.

Idempotency

The operation must satisfy:

```
Apply(part) + Apply(part)
=
same drawing as Apply(part)
```

not:

```
new hatch objects
+ duplicate geometry
+ cumulative angle/spacing changes
```

For every hatch compute a canonical desired-style record:

```
{
  "ownerKey": "TOP/SUB_A/PLATE_01|10024567|C",
  "ruleSet": "NX2306_SECTION_HATCH_V4",
  "rule": "MAT_STEEL",
  "pattern": "STEEL_STD",
  "angle": 45.0,
  "spacing": 2.0,
  "scale": 1.0
}
```

Canonicalize it and hash it:

```
SHA-256(
  ownerKey |
  ruleset |
  rule |
  pattern |
  angle |
  spacing |
  scale
)
```

Then:

```
Existing hash == desired hash
    → NO OP

Managed hatch, hash differs because rule changed
    → UPDATE

Managed hatch, stored previous hash differs from
current visible style
    → probable MANUAL EDIT
    → conflict unless overwrite policy allows it

MBD_HATCH_LOCK == TRUE
    → preserve current hatch
```

```
No management attributes
    → classify as unmanaged; review before taking ownership
```

Crucially, **the hash itself is not the owner mapping**. Ownership is recalculated each time from section topology so the system survives section-plane movement and component repositioning.

Conflict model

Do not merely log strings. Use typed conflict states:

Conflict	Meaning	Default action
`NO_OWNER`	Hatch cannot be mapped to a component	Leave unchanged
`MULTIPLE_OWNERS`	Two or more candidate section solids remain equally plausible	Leave unchanged
`MISSING_MATERIAL`	Material rule requested but no recognized material	Try P/N fallback only if policy permits
`MISSING_PART_NUMBER`	No P/N available	Use material if authoritative; otherwise conflict
`RULE_COLLISION`	Two equal-authority rules produce different styles	Leave unchanged
`UNKNOWN_PATTERN`	Configured crosshatch pattern/ file cannot be resolved	Leave unchanged
`MANUAL_EDIT_DETECTED`	Managed hatch appearance differs from last automation state	Require review
`LOCKED`	User/company lock prevents automatic change	Preserve
`STALE_VIEW`	Drawing section is out of date	Update before mapping
`COINCIDENT_GEOMETRY`	Geometry cannot uniquely distinguish overlapping bodies	Require manual resolution
`SUPPRESSED_OWNER`	Recorded owner no longer active in current arrangement	Rebuild mapping
`TEAMCENTER_ATTRIBUTE_MISSING`	Expected mapped property is absent	Report mapping/data issue

A key acceptance principle should be:

an ambiguous region must never be silently assigned to the "closest" component.

Review UI

Use a Block UI Styler dialog or equivalent NX dialog with one row per hatch region:

Status	View	Region	Component path	P/N	Material	Current	Proposed	Rule	Confidence
Ready	SECTION A-A	R01	TOP/HSG	10024567	STEEL	General 45°	Steel 45°	MAT_STEEL	High
Ready	SECTION A-A	R02	TOP/SUB/ INSERT	20078412	AL6061	General 45°	Al 135°	MAT_AL	High
Conflict	SECTION A-A	R03	—	—	—	General	—	—	Multiple
Locked	SECTION B-B	R07	TOP/BRG	BRG-6204	BRONZE	Custom	Custom	PN_BRG	High

Recommended commands:

```
[Analyze]
[Highlight Region]
[Highlight Component]
[Show Attribute Sources]
[Accept Proposed]
[Keep Manual]
[Apply Override]
[Lock]
[Apply Selected]
[Validate All]
[Export Audit]
```

Selecting a row should highlight both the hatch and its occurrence component. The engineer must be able to see **why** a rule fired:

```
Proposed style: STEEL_STD / 45° / 2.0 mm

Resolved from:
  MATERIAL_ID = MAT-STEEL-4140
  Source = prototype NX attribute
  Rule = MAT_STEEL
  Ruleset = V4
```

```
P/N:
  DB_PART_NO = 10024567
  Source = Teamcenter mapping

Adjacency adjustment:
  none
```

Teamcenter integration strategy

Keep Teamcenter out of the geometry-mapping layer.

The dependency should be:

```
Geometry → Component
              ↓
        AttributeResolver
          /          \
     NX local      TC-mapped
      attrs          attrs
          \          /
           normalized metadata
                  ↓
              Rule engine
```

Where managed material data is available through Teamcenter Integrated Material Management, Siemens states that the material assignment can be synchronized with NX. That makes Teamcenter a viable material gold source, but your application should consume the synchronized/mapped value through an adapter rather than embedding Teamcenter schema names throughout the code. [9]

For BOM-line or occurrence properties, site mapping matters. Siemens community material for NX 2306-era integration discusses exposing Teamcenter BOMLine properties through NX attribute mapping, while Siemens' NX 2306 mapping troubleshooting confirms that mapped attribute behavior is configuration-sensitive. [23]

A clean interface is:

```
public interface IEngineeringMetadataProvider
{
    AttributeValue? GetPartNumber(
        Component occurrence,
        Part prototype);

    AttributeValue? GetRevision(
        Component occurrence,
        Part prototype);
```

```
    AttributeValue? GetMaterial(
        Component occurrence,
        Part prototype);

    AttributeValue? GetExplicitHatchOverride(
        Component occurrence,
        Part prototype);
}
```

Implement:

```
NxAttributeMetadataProvider
TeamcenterMappedMetadataProvider
CompositeMetadataProvider
```

Do not perform direct server queries for every hatch. Cache metadata by prototype part/revision and add occurrence-specific values separately.

Performance strategy for large assemblies

The biggest performance mistake would be:

```
for each hatch
    for each assembly component
        for each face
            ray cast
```

Instead:

```
NX SectionView
     ↓
section solids only
     ↓
candidate components only
```

Then use caches:

```
Occurrence index:
  Component.Tag → full path / prototype

Prototype metadata:
  Part revision → P/N / material
```

```
Section topology:
  section edge → section solid → component

Rule cache:
  normalized material/P/N → resolved style

Hatch state:
  hatch → owner/style hash
```

For a large drawing:

1. **Build the component tree once.** `GetChildren()` is available on the SC2306 component API. [6]
2. **Restrict geometry to sectioned solids.** Do not scan every body in the full assembly.
3. **Resolve attributes once per prototype revision.** Ten occurrences of one bolt should not trigger ten identical material lookups.
4. **Resolve occurrence attributes separately.** Cache by stable occurrence path.
5. **Use bounding boxes before expensive topology/ray tests.**
6. **Use direct section-edge ownership before geometry fallback.**
7. **Only open a `HatchBuilder` when desired style differs.**
8. **Commit updates as a batch under one undo mark.**
9. **Defer reporting/UI refresh until analysis is complete.**
10. **Keep NX API calls serialized in the NX execution context; pure rule matching/hash computation can be isolated from NX-object access.**

For initial performance qualification, create representative benchmarks rather than optimizing against a toy assembly.

Suggested project targets—not Siemens performance claims—are:

```
200 section regions:
  analyze + resolve < 5 s

1,000 section regions:
  analyze + resolve < 15 s

2,000 section regions:
  analyze + resolve < 30 s

Repeat run with no rule/model changes:
  substantially faster due to caches
  and zero unnecessary hatch commits
```

Measure these on your actual engineering workstation, Teamcenter network, and assembly-loading configuration before making them release requirements.

Test matrix and acceptance criteria

Test-case matrix

ID	Assembly / drawing condition	Expected mapping	Expected hatch behavior	Primary validation
T01	One steel block, one section region	One hatch → one component	Steel pattern applied	Exact owner, exact rule
T02	Steel + aluminum components adjacent	Two unique owners	Different material patterns	No cross-assignment
T03	Two adjacent steel components	Two unique owners	Same base pattern, deterministic angle/spacing variant if policy enabled	Readability + repeatability
T04	Concentric steel housing and bronze bushing	Outer/inner loops map separately	Steel vs bronze patterns	Inner-loop/topology handling
T05	Nested subassembly three levels deep	Leaf part identified with complete path	Leaf material/P/N rule	Parent-chain correctness
T06	Same prototype used in four occurrences	Each region maps to its occurrence	Same prototype style unless occurrence override exists	No prototype/occurrence confusion
T07	Same prototype with one instance override	Override occurrence resolved separately	One region differs according to override	Instance attribute precedence
T08	Components overlap in drawing projection but only one is intersected by section plane	Section-solid owner only	Correct intersected component style	No projection-only false positive
T09	Two coincident cut faces from different components	Ambiguous unless topology distinguishes	No automatic style if unresolved	Conflict, no silent guess

ID	Assembly / drawing condition	Expected mapping	Expected hatch behavior	Primary validation
T10	Section plane moved across new components	New section topology rebuilt	New/changed hatches styled correctly	No stale owner keys
T11	Component translated/ rotated	Updated view retains native boundary associativity	Same component rule reapplied	No manual detached hatch
T12	Component suppressed in active arrangement	Suppressed occurrence absent	Its hatch disappears/not processed	Arrangement correctness
T13	Material missing, P/N rule exists	P/N fallback used if allowed	P/N style	Correct fallback source
T14	Material and P/N rules conflict at equal authority	Conflict	No automatic modification	`RULE_COLLISION`
T15	Invalid pattern name	Owner still known	Existing hatch preserved	`UNKNOWN_PATTERN`
T16	Manually edited automation-managed hatch	Owner known	Conflict or overwrite per policy	Manual-edit detection
T17	`MBD_HATCH_LOCK=TRUE`	Owner known	No style modification	Lock honored
T18	Teamcenter-managed part with mapped `DB_PART_NO`	Correct P/N consumed	Rule based on managed P/N/ material	Attribute source logged
T19	Teamcenter material updated between revisions	Correct current loaded revision value	New style if rule changes	Revision-aware cache
T20	Drawing section with holes/ islands	Same component can own multiple disjoint regions	Consistent style across its regions	Loop/island handling
T21	Multiple section views of same component	All regions map independently	Controlled propagation only	No unintended cross-view edit

ID	Assembly / drawing condition	Expected mapping	Expected hatch behavior	Primary validation
T22	Stale/out-of-date drawing view	Processing blocked or update requested	No mapping on stale topology	`STALE_VIEW`
T23	500+ occurrence assembly, 200 cut bodies	Correct owners	Batch update	Performance target
T24	5,000+ occurrence assembly, small cut set	Only cut-set candidates evaluated geometrically	Correct results without full-face brute force	Scaling behavior
T25	3D section feasibility model	Region/body mapping demonstrated	Style mutation proven or limitation documented	Go/no-go for 3D adapter

Ownership acceptance criteria

The production release should meet:

100% correct ownership for all unambiguous certified test cases.

A 99% "best guess" is not acceptable if the remaining 1% receives the wrong manufacturing-section representation. An unresolved hatch should become a visible exception.

For every processed region:

```
Exactly one of:

PASS:
    owner = one component
    confidence >= approved threshold

CONFLICT:
    candidate owners > 1

UNRESOLVED:
    owner = none
```

There must be **zero silent ambiguous mappings**.

Associativity acceptance criteria

After the original drawing is styled:

1. move a component;
2. update the drawing;
3. move the section line;
4. update again;
5. suppress/unsuppress an occurrence;
6. rerun the hatch manager.

The solution passes only when:

```
NX section boundaries regenerate natively
no automation-created detached boundary curves exist
no duplicate hatch regions are created
current topology resolves to current components
styles are reconciled to the current components
obsolete owner records do not control new geometry
```

NX's drawing workflow itself supports detecting/updating drawings after design changes, so the automation should operate after NX has brought the native view current rather than trying to substitute for the view-update engine. [22]

Idempotency acceptance criteria

Run:

```
Analyze → Apply → Save
Analyze → Apply → Save
Analyze → Apply → Save
```

with no engineering changes.

Expected:

```
Hatch count                    unchanged
Section boundary count         unchanged
Hatch style                    unchanged
Owner mapping                  unchanged
Rule IDs                       unchanged
Second/third apply mutations   zero
```

Then modify one material attribute and rerun.

Expected:

```
Only regions owned by affected occurrence/prototype change
Unrelated hatch objects remain untouched
Audit rule/style hash updates exactly once
```

Teamcenter acceptance criteria

The Teamcenter-enabled implementation should:

- correctly identify which value came from Teamcenter/NX mapping versus a local custom attribute;
- cache prototype metadata by revision rather than part number alone;
- support occurrence/BOM-line properties when your integration maps them into NX;
- never overwrite Teamcenter-controlled P/N, revision, or material identity merely to implement hatching;
- report absent or inconsistent mapped properties rather than fabricating values.

Teamcenter's NX integration and material-management tooling are explicitly designed around mapped/synchronized CAD and material information, but the site's actual schema and mapping remain configuration-specific. [24]

Phased implementation plan and deliverables

Feasibility spike

Goal: prove that NX 2306 exposes enough native data to map one section-generated hatch to an occurrence body without geometric guessing.

Build a small assembly:

```
TOP
├── PLATE_STEEL
├── BLOCK_ALUMINUM
└── BUSH_BRONZE
```

Create one assembly section view.

Implement:

```
Enumerate SectionView
Enumerate Annotations.Hatch
Get hatch.GetViews()
Get hatch associativities
Ask section solids
```

```
Ask section edges
Ask original solid
Body.OwningComponent
Edit existing HatchBuilder
```

Deliverables:

```
NX2306_SectionTopologyProbe.dll
API behavior matrix
recorded NX2306 crosshatch journal
one annotated test drawing
license/configuration report
```

Go/no-go requirement: at least one native section-generated hatch can be modified without rebuilding its boundary, and section-solid topology can be traced back to the correct occurrence.

The UF_DRAW topology sequence should be explicitly revalidated here because the publicly visible journal example is older even though the NX 2306 component/hatch APIs themselves are documented in SC2306. [25]

Production ownership engine

Implement:

```
SectionTopologyIndex
OccurrenceIndex
Nested component path generation
Direct association mapper
Section-edge mapper
Region loop reconstruction
Multi-point containment fallback
Ray-intersection verifier
Confidence scoring
```

Deliverables:

```
SectionTopology.dll
OwnershipResolver.dll
Mapping diagnostics
T01–T12 automated regression models
```

At the end of this phase, material rules are unnecessary. The system must first prove:

"This hatch belongs to this component."

Attribute and rules engine

Implement:

```
IEngineeringMetadataProvider
NX local attributes
Teamcenter-mapped attributes
instance attributes
prototype attributes
normalization tables
material rules
part-number rules
precedence
conflict detection
adjacency presentation variants
```

Deliverables:

```
MetadataResolver.dll
HatchRules.json
schema + validation
attribute mapping configuration
Teamcenter test report
```

For managed NX, coordinate this phase with the Teamcenter administrator because Siemens' own documentation shows that NX/Teamcenter attribute behavior is mapping-dependent, while managed material synchronization can be provided through Teamcenter material-management integration. [19]

Hatch writer and idempotency

Implement:

```
HatchStyleReader
HatchStyleWriter
Crosshatch-file resolver
style hash
managed-hatch metadata
manual edit detection
lock behavior
one undo transaction
update/reconcile operation
```

The key implementation primitive is editing an existing `Hatch` via `CreateHatchBuilder` and the native hatch-fill settings. [1]

Deliverables:

```
HatchWriter.dll
idempotency regression suite
section-movement test suite
manual-edit/lock tests
audit JSON/CSV
```

Review UI and release validation

Implement a Block UI Styler interface with:

```
View / Region
Owner component
P/N
Material
Source attributes
Current style
Proposed style
Rule
Confidence
Conflict
```

Add commands:

```
Highlight
Accept
Keep manual
Override
Lock
Apply
Validate
Export report
```

Deliverables:

```
Section Hatch Manager UI
release validator
```

```
user guide
support diagnostics package
```

A release validator should block a "green" result when any of the following remain:

```
unmapped hatch
multiple-owner hatch
unknown hatch pattern
rule conflict
stale drawing view
unapproved manual edit
missing mandatory material/P/N
```

Large-assembly hardening

Run the full test matrix using production-scale assemblies and actual Teamcenter connectivity.

Instrument timings separately for:

```
view topology extraction
component resolution
attribute resolution
geometric fallback
rule evaluation
hatch writes
NX update
```

Optimize only the measured bottlenecks.

Deliverables:

```
performance benchmark report
cache strategy
memory profile
large-assembly regression suite
failure/recovery tests
deployment package
```

3D section adapter

Treat 3D sectional display as a separate qualification project, sharing everything after ownership extraction:

```
3D Section Adapter
       ↓
cut face / body ownership
       ↓
same metadata resolver
       ↓
same material/P/N rules
       ↓
3D-specific appearance writer
```

The deliverable should answer two concrete NX 2306 questions before production scope is committed:

```
What NX object represents the generated 3D cut-region hatch/fill?

Can its style be changed through supported NX Open APIs
without replacing the native section geometry?
```

If the answer to the second question is no for NX 2306, keep the 3D functionality at **classification/validation/highlighting** and deploy automatic styling only for drafting section views. That is preferable to implementing an unsupported detached-graphics workaround.

Recommended production architecture

The final codebase should look approximately like this:

```
Company.Nx.SectionHatch/
│
├── Entry/
│   ├── SectionHatchCommand.cs
│   └── NxSessionContext.cs
│
├── Views/
│   ├── ISectionAdapter.cs
│   ├── DraftingSectionAdapter.cs
│   └── ModelSectionAdapter.cs
│
├── Topology/
│   ├── SectionTopologyIndex.cs
│   ├── SectionSolidResolver.cs
│   ├── RegionLoopBuilder.cs
│   ├── RayIntersectionVerifier.cs
│   └── OwnershipConfidence.cs
│
├── Assembly/
```

```
│   ├── OccurrenceIndex.cs
│   ├── ComponentPath.cs
│   └── PrototypeResolver.cs
│
├── Metadata/
│   ├── IEngineeringMetadataProvider.cs
│   ├── NxAttributeProvider.cs
│   ├── TeamcenterMappedProvider.cs
│   └── MetadataNormalizer.cs
│
├── Rules/
│   ├── HatchRuleEngine.cs
│   ├── MaterialRule.cs
│   ├── PartNumberRule.cs
│   ├── AdjacencyResolver.cs
│   └── HatchRules.schema.json
│
├── Authoring/
│   ├── HatchStyleReader.cs
│   ├── HatchStyleWriter.cs
│   ├── CrosshatchFileResolver.cs
│   └── ManagedHatchState.cs
│
├── Validation/
│   ├── MappingValidator.cs
│   ├── RuleValidator.cs
│   ├── AssociativityValidator.cs
│   └── ReleaseReport.cs
│
├── UI/
│   ├── SectionHatchReviewDialog.cs
│   └── HighlightService.cs
│
└── Config/
    ├── hatch-rules.json
    └── attribute-map.json
```

The central design boundary is:

```
NX geometry/topology
       ↓
ComponentIdentity
       ↓
EngineeringMetadata
       ↓
HatchStyleSpecification
```

```
       ↓
NX hatch style writer
```

That separation is what makes the solution maintainable. It allows Teamcenter mappings, material conventions, part-number rules, crosshatch libraries, and even the section-view API adapter to change independently.

For **NX 2306 specifically**, I would approve development on the following basis:

Use C# as the production language; make drafting assembly sections the first supported view type; resolve ownership from NX section-solid topology, not screen pixels; read Teamcenter information through a configurable metadata adapter; modify existing native `Hatch` objects rather than creating replacement geometry; make all ambiguous mappings reviewable rather than guessed; and rebuild owner/style reconciliation after every relevant view update. The SC2306 APIs expose the required assembly occurrence and hatch-management foundations, while Siemens' crosshatch and Teamcenter documentation supports the component-oriented and mapped-attribute parts of this architecture. [26]

---

[1] [16] [17] Class HatchCollection
https://nxopencsdocumentation.thescriptingengineer.com/SC2306/NXOpen.Annotations.HatchCollection.html

[2] [5] Siemens Digital Industries Software Community
https://community.sw.siemens.com/s/article/crosshatching-enhancements-to-drawing-in-nx-11-0-1

[3] [6] [11] [18] [26] Class Component
https://nxopencsdocumentation.thescriptingengineer.com/SC2306/NXOpen.Assemblies.Component.html

[4] [25] BOM balloons without a parts list | NX Journaling
https://nxjournaling.com/content/bom-balloons-without-parts-list

[7] Siemens Digital Industries Software Community
https://community.sw.siemens.com/s/question/0D5Vb00001EFQkxKAH/nx-license-bundle-nx30104-includes-the-necessary-rights-and-components-to-develop-build-and-commercially-deploy-custom-pluginsaddons-for-siemens-nx-using-the-cnet-api-framework-nx-open

[8] [19] [24] Siemens Digital Industries Software Community
https://community.sw.siemens.com/s/article/TC-NX-attribute-mapping-displays-different-values-for-an-attribute

[9] The Right Material Knowledge for Successful Products - Teamcenter
https://blogs.sw.siemens.com/teamcenter/the-right-material-knowledge-for-successful-products/

[10] Deep CAD search | Teamcenter
https://blogs.sw.siemens.com/teamcenter/cad-search-teamcenter/

[12] [20] Class HatchFillSettingsBuilder
https://nxopencsdocumentation.thescriptingengineer.com/NX12/NXOpen.Annotations.HatchFillSettingsBuilder.html

[13] Class Annotation
https://nxopencsdocumentation.thescriptingengineer.com/NX2022_1/NXOpen.Annotations.Annotation.html

14 Siemens Digital Industries Software Community
https://community.sw.siemens.com/s/question/0D54O000061xKl6SAE/nxopenc-how-to-tell-whether-random-points-on-the-selected-face-or-not

15 Siemens Digital Industries Software Community
https://community.sw.siemens.com/s/question/0D54O000061xFk0SAE/get-intersection-point-on-face-from-vector-direction-vectorprojection

21 Class Pmi
https://nxopencsdocumentation.thescriptingengineer.com/SC2306/NXOpen.Annotations.Pmi.html?utm_source=chatgpt.com

22 Designcenter NX | How to | Creating Technical Drawings
https://blogs.sw.siemens.com/designcenter/designcenter-nx-how-to-technical-drawings/

23 Siemens Digital Industries Software Community
https://community.sw.siemens.com/s/question/0D5Vb0000053AHmKAM/accessing-teamcenter-form-and-bomline-properties-tcnxdesignattributegroupproperties